



Testing: Unit & Integration

Testing Pyramid · @WebMvcTest · @DataJpaTest · TestContainers



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday · Online

TIME

3:00 – 5:30 PM

DURATION

2.5 Hours · Phase 2

Phase 2 — Quality & Deployment



Today: the platform runs in containers — but do we trust what's inside? Testing is the answer.

Full 29-session roadmap: see Course Kickoff deck

S9: Platform Runs in Containers. But Do We Trust It?

WHAT S9 GAVE US

- docker compose up -d starts the full platform reproducibly
- Every service runs in its own container with the right JRE
- Environment vars control config — no more localhost mismatch
- The platform is consistent. But is the code inside it correct?

THE QUESTION WE HAVEN'T ANSWERED

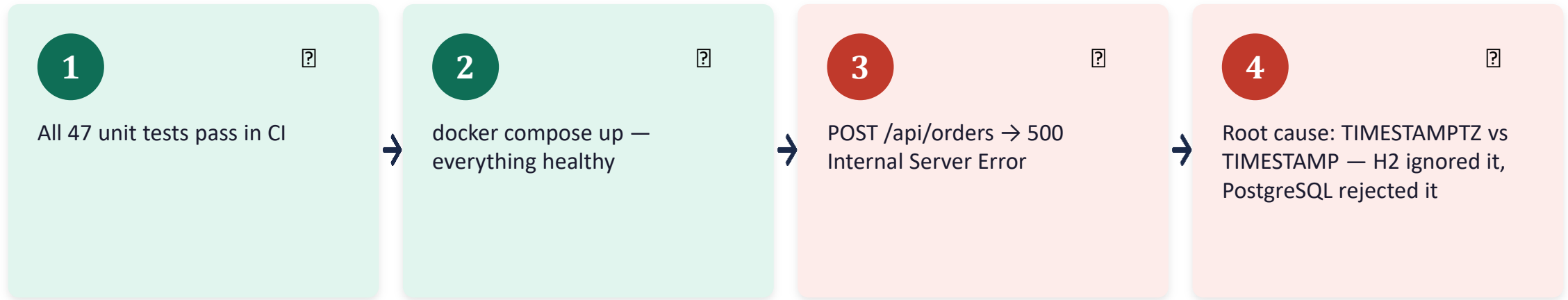
- If Order Service calls Inventory and gets a 500 — is it a network bug or a business logic bug?
- If Product API returns 200 but with wrong field names — who catches that before deployment?
- A Saga compensation that triggers at the wrong state — how do we know before a customer sees it?

Answer: Tests. Tests that cover the right layers. Tests that run fast AND catch real problems.

TODAY'S AGENDA

Testing Pyramid · Unit Tests with JUnit 5 + Mockito · @WebMvcTest slice · TestContainers for real PostgreSQL

“All Tests Green” — But Production Fails Anyway



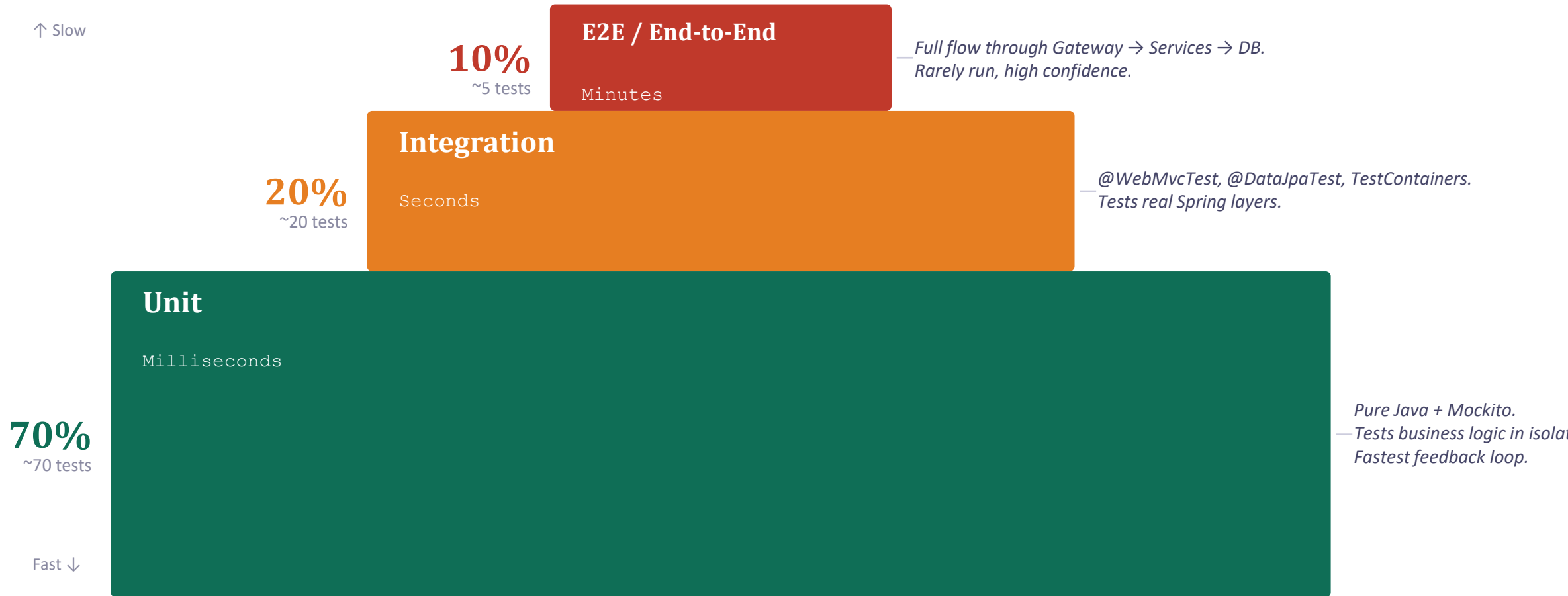
Why did this happen?

- Unit tests mocked the repository — never touched a real database
- `@DataJpaTest` used H2 in-memory — H2 silently accepts TIMESTAMPTZ as TIMESTAMP
- PostgreSQL enforces the type strictly — the mismatch only appears in production
- The tests passed because they tested the wrong thing against the wrong database

The Wrong Test for the Wrong Layer

Production Problem	Wrong Test Used	Why It Still Passed	? Right Test
Business logic bug (wrong discount formula)	<code>@DataJpaTest</code>	Loads Spring Data + DB — slow and irrelevant for pure logic	Plain JUnit unit test + @Mock repository
Controller returns wrong HTTP status for missing product	<code>Full Spring Boot</code> <code>@SpringBootTest</code>	Starts entire app including DB — overkill, 30s startup	@WebMvcTest (loads only MVC layer)
PostgreSQL-specific SQL fails at runtime	<code>@DataJpaTest</code> + <code>H2</code>	H2 doesn't enforce PostgreSQL type rules — false green	TestContainers (real PostgreSQL)

Testing Pyramid — Test at the Right Level



Unit / Integration / E2E — Compared

Layer	Scope	Speed	Tools	Spring Slice
Unit	Verify: Business logic in isolation — no I/O One class/method in isolation	< 5 ms	JUnit 5 + Mockito + @ExtendWith	None (plain Java)
Integration	Verify: Layer interactions: controller, JPA repo, service+DB One Spring layer + real collaborators	< 5 s	Spring Test + TestContainers @WebMvcTest, @DataJpaTest	@WebMvcTest @DataJpaTest
E2E	Verify: Full flow across services Full request through Gateway → DB → response	Minutes	RestAssured, Playwright Covered in Phase 3 (Session 23 — load testing)	@SpringBootTest webEnvironment=RANDOM_PORT

Rule of thumb: 70% Unit · 20% Integration · 10% E2E. More integration tests than in monoliths — because service contracts must be verified.

Why Microservices Need More Integration Tests

Monolith

- One process — compiler catches inter-module type mismatches at build time
- One DB transaction — rollback is atomic, no Saga compensation to test
- Internal calls are method calls — no HTTP serialization boundary
- → Unit tests alone cover most contract surface

Microservices

- Each service is a separate process — JSON deserialization is a contract boundary
- Order → Inventory via HTTP: a field rename breaks the caller silently
- Saga compensation depends on event structure — must be tested with real Kafka or mock
- → Integration tests are essential, not optional

Today's focus: layers Unit (JUnit 5/Mockito) and Integration (@WebMvcTest + TestContainers). E2E deferred to Phase 3.

@ParameterizedTest — One Method, Many Inputs

BEFORE — DUPLICATED, BRITTLE

```
@Test void calcDiscount_silver() {
    assertEquals(5, svc.calcDiscount("SILVER"));
}
@Test void calcDiscount_gold() {
    assertEquals(10, svc.calcDiscount("GOLD"));
}
@Test void calcDiscount_platinum() {
    assertEquals(15, svc.calcDiscount("PLATINUM"));
}
```

AFTER — ONE TEST, COVERS ALL CASES

```
@ParameterizedTest
@CsvSource({
    "SILVER, 5",
    "GOLD, 10",
    "PLATINUM,15"
})
void calcDiscount(String tier, int expected) {
    assertEquals(expected,
        svc.calcDiscount(tier));
}
```

💡 Three test cases for the price of one. Add a new tier? Add one CSV row. No copy-paste. Failures show exactly which input failed.

ArgumentCaptor — Verify What Was Actually Passed, not just that it was called

```
// We want to verify the EXACT object that was saved — not just that save() was called
@Test
void createOrder_savesOrderWithPendingStatus() {
    // Given
    ArgumentCaptor<Order> orderCaptor = ArgumentCaptor.forClass(Order.class);
    when(inventoryClient.reserveStock(any())) .thenReturn(true);

    // When
    orderService.createOrder(new OrderRequest("PROD-1", 3, new BigDecimal("99.99")));

    // Then: verify the EXACT object saved to repository
    verify(orderRepository).save(orderCaptor.capture());
    Order saved = orderCaptor.getValue();
    assertEquals(OrderStatus.PENDING, saved.getStatus());
    assertEquals("PROD-1",          saved.getProductId());
    assertEquals(3,                  saved.getQuantity());
}
```



Why not verify(repo).save(order)?

verify() only checks that save() was called — not what was passed.

ArgumentCaptor lets you assert the actual field values of the saved object. Critical for checking business rules are applied before persistence.

@Spy vs @Mock — The Concrete Rule

@Mock

What it is:

A complete fake. All methods return default values (null / 0 / false) unless stubbed.

Use when:

*You don't care about the real implementation.
For dependencies with side effects (DB, HTTP).
You only want to control what it returns.*

```
// @Mock replaces the whole object — no real method runs
@Mock ProductRepository repo;

// all methods return null/empty by default unless stubbed

when(repo.findById(1L))
    .thenReturn(Optional.of(product));
```

@Spy

What it is:

A real object with optional method overrides. Calls real methods unless explicitly stubbed.

Use when:

*You want real behavior for most methods, but override one specific method.
Never @Spy an interface — only concrete classes.
@Spy on an abstract class requires a no-arg constructor.*

```
// @Spy wraps a REAL object — only stubbed methods are replaced
@Spy DiscountService discount;

// Real calcDiscount() runs for most inputs, but you can stub
// specific cases:

doReturn(0).when(discount)
    .calcDiscount("TEST");
```

The Right Tool for Each Layer

Annotation	Loads	Excludes	Best For
<code>@WebMvcTest</code>	DispatcherServlet, Controllers, @ControllerAdvice, Filters, MockMvc	Services, Repositories, DB, full Spring context	Test HTTP layer: request/response, status codes, JSON format, validation
<code>@DataJpaTest</code>	JPA repositories, EntityManager, H2 by default	Web layer, Controllers, Services, security, non-JPA beans	Test repository queries (non-PG-specific), @Query methods, relationships
<code>@JsonTest</code>	Jackson ObjectMapper only	Everything else	Test JSON serialization of your DTOs
<code>@SpringBootTest</code>	Full application context	Nothing — everything starts	Integration: when two or more layers interact Last resort / E2E. Slowest. Use only when slice isn't enough

🔗 Use the narrowest slice possible. @SpringBootTest is ~10× slower than @WebMvcTest and loads things you don't need for most tests.

@DataJpaTest + H2 vs TestContainers + PostgreSQL



If: Speed critical, no DB-specific queries?

→ `@DataJpaTest` + H2

H2 boots in ~100ms vs ~3s for a real container. Fine for simple CRUD — findById, save, delete.



If: PostgreSQL-specific queries (JSON operators, arrays, TIMESTAMPTZ)?

→ `TestContainers` (required)

H2 silently accepts PG-specific types. The bug only appears in production. Must test against real PostgreSQL.



If: CI/CD pipeline, Docker available in the build agent?

→ `TestContainers` (preferred default)

Same result as production — no false greens. Docker-in-Docker or GitHub Actions docker socket is the norm. This is our course standard from Lab 9A onwards.



If: Team learning testing for the first time?

→ Start with `@DataJpaTest` — introduce `TestContainers` in Lab 9B

Lower setup friction for first contact. Once the pattern is understood, swap to `TestContainers` for realism.

Add test dependencies to pom.xml

```
dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
  <!-- Includes: JUnit 5, Mockito, MockMvc, AssertJ, Hamcrest -->
</dependency>

<!-- TestContainers – for PostgreSQL integration tests (Part 5) -->
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>

<!-- TestContainers BOM in dependencyManagement: -->
<!-- <artifactId>testcontainers-bom</artifactId> <version>1.19.3</version> -->
```

ProductControllerTest — Class Setup + Happy Path

```
// @WebMvcTest - controller isolation with MockMvc

@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;
    @MockBean ProductService productService; // Mockito mock injected into slice

    @Test
    @DisplayName("GET /api/products/{id} returns 200 with product JSON")
    void getProduct_found_returns200() throws Exception {
        // Given
        var product = new Product(1L, "Laptop", new BigDecimal("999.99"));
        when(productService.findById(1L)).thenReturn(Optional.of(product));
        // When + Then
        mockMvc.perform(get("/api/products/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Laptop"))
            .andExpect(jsonPath("$.price").value(999.99));
    }
}
```

ProductControllerTest — Error Cases

```
@Test
@DisplayName("GET /api/products/{id} returns 404 when not found")
void getProduct_notFound_returns404() throws Exception {
    when(productService.findById(99L)).thenReturn(Optional.empty());
    mockMvc.perform(get("/api/products/99"))
        .andExpect(status().isNotFound());
}

@Test
@DisplayName("POST /api/products with missing name returns 400")
void createProduct_missingName_returns400() throws Exception {
    var badRequest = new CreateProductRequest(null, new BigDecimal("50"));
    mockMvc.perform(post("/api/products")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(badRequest)))
        .andExpect(status().isBadRequest());
}

// No DB, no Spring Data — just the web layer. Runs in < 2 seconds.
```

@WebMvcTest loads ONLY: DispatcherServlet + your Controller + Jackson. No repositories, no DB.
The MockMvc call never touches a socket — it's an in-memory dispatch.

mvn test — Web Layer in < 3 Seconds

```
$ mvn test -pl services/product-service -Dtest=ProductControllerTest
```

```
[INFO] -----  
[INFO]  T E S T S  
[INFO] -----  
[INFO] Running ProductControllerTest  
Starting MockMvcBuilderConfiguration...  
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  
[INFO]   Time elapsed: 2.341 s  
[INFO] BUILD SUCCESS
```

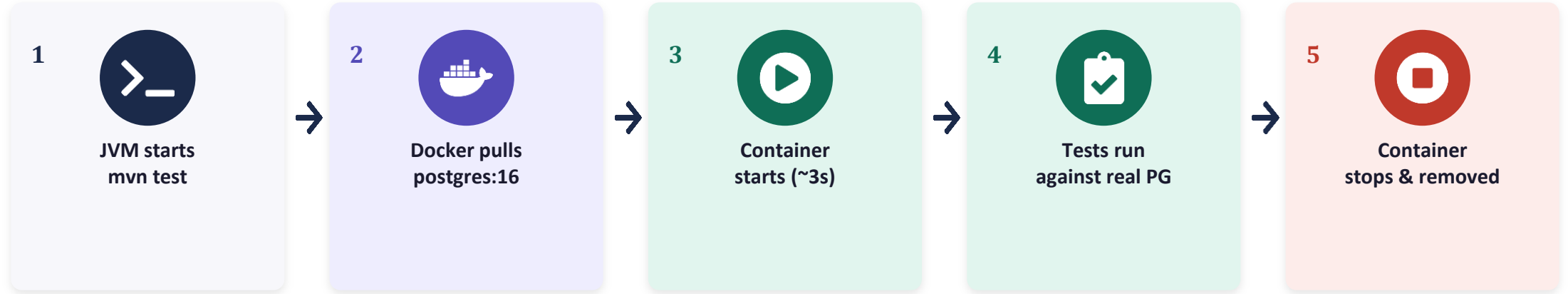
What @WebMvcTest loaded:

- ProductController ✓
- Jackson ObjectMapper ✓
- Validation filters ✓

What it did NOT load:

- ProductService (mocked ✓)
- ProductRepository (never loaded)
- PostgreSQL / any DB

How TestContainers Works — Container Lifecycle



WHAT IT GIVES YOU

- Real PostgreSQL — same version as production
- Every test run gets a clean database
- Port is randomised at startup — no port clashes in CI
- `@DynamicPropertySource` patches `spring.datasource.url` automatically

THE ONLY TRADE-OFF

- ~3 s for container startup — run once per test class via `@Container`
- Docker must be running in CI build agent
- First run downloads the image (~200 MB) — cached after that
- Still 5–10× faster than a `@SpringBootTest` with full context

ProductRepositoryIntegrationTest — Container Setup

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = NONE) // don't replace with H2
@Testcontainers
class ProductRepositoryIntegrationTest {

    @Container // started once, shared across all tests
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16")
            .withDatabaseName("testdb")
            .withUsername("test")
            .withPassword("test");

    @DynamicPropertySource // auto-patches spring.datasource.*
    static void registerProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired ProductRepository productRepository;
```

ProductRepositoryIntegrationTest — The Tests

```
@Test
void save_andFindById_roundTrip() {
    // Given
    var p = new Product(null, "Laptop", new BigDecimal("999.99"));
    // When
    var saved = productRepository.save(p);
    var found = productRepository.findById(saved.getId());
    // Then
    assertTrue(found.isPresent());
    assertEquals("Laptop", found.get().getName());
    assertEquals(new BigDecimal("999.99"), found.get().getPrice());
}

@Test
void findByPriceLessThan_returnsMatchingProducts() {
    productRepository.saveAll(List.of(
        new Product(null, "Mouse", new BigDecimal("29.99")),
        new Product(null, "Monitor", new BigDecimal("399.99"))));
    var cheap = productRepository.findByPriceLessThan(new BigDecimal("50"));
    assertEquals(1, cheap.size());
    assertEquals("Mouse", cheap.get(0).getName());
}
} // Container stops automatically after all tests in this class
```

TestContainers Log — Real PostgreSQL, Real Results

```
$ mvn test -Dtest=ProductRepositoryIntegrationTest
```

```
❑ Starting postgres:16 container...
```

```
✓ Container postgres:16 started in PT3.241S
```

```
✓ jdbc:postgresql://localhost:52341/testdb
```

```
[INFO] Running ProductRepositoryIntegrationTest
```

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO] Time elapsed: 4.812 s
```

```
❑ Stopping postgres:16 container...
```

```
[INFO] BUILD SUCCESS
```

What happens when mvn test runs this:

1. TestContainers pulls postgres:15-alpine (first run only — cached after that)
2. Starts a real PostgreSQL container on a random host port
3. @DynamicPropertySource injects
jdbc:postgresql://localhost:<random_port>/productdb_test
4. Spring Boot creates schema (Flyway/Hibernate DDL) against real PostgreSQL
5. Tests run — real SQL, real constraints, real behaviour
6. Container stops automatically after all tests complete

4.8 seconds total including container start, Flyway migration, 2 tests, container stop.

- Container starts ONCE per test class ('static') — not before each @Test
- Next run: Docker cache hit — container starts in < 0.5 s



LAB 9A

Write Tests for Product Service

🕒 50 minutes · Target: product-service · Grade: Feature 70% + Tests 20% + Quality 10%

1

Unit: @ParameterizedTest

12 min

Write ProductServiceTest. Test calcDiscount() for SILVER/GOLD/PLATINUM with @CsvSource. Add ArgumentCaptor test: verify createProduct() saves with correct status.

2

@WebMvcTest Controller

18 min

Write ProductControllerTest. Cover: GET 200, GET 404, POST 201, POST 400 (missing name). Use @MockBean for ProductService. Assert jsonPath on all success responses.

3

TestContainers Repository

20 min

Write ProductRepositoryIntegrationTest with postgres:16 container. Test: save-and-find round-trip, findByPriceLessThan(). Add @AutoConfigureTestDatabase(replace=NONE).

Verified Behaviour — Unit + Integration Test Coverage

🔍 Capability Added

Verified Behaviour

Product Service: @ParameterizedTest + @WebMvcTest + TestContainers.
Business logic, HTTP layer, and DB queries each tested at the right level.

🔍 What's Now Tested

- calcDiscount() — unit, all tiers via @ParameterizedTest
- GET/POST /api/products — @WebMvcTest, 4 cases
- ProductRepository — TestContainers + real PostgreSQL

Enterprise E-Commerce Platform — After Session 10



Docker



Testing I



Testing II



Saga Orch



CI/CD



GitOps



K8s Core



K8s Adv

HOMEWORK • BEFORE SESSION 11

- Run mvn test — all 3 test classes green before S11
- Pre-reading: Consumer-Driven Contract Testing (Pact) — link in Session Pack



DAILY QUIZ

8 Questions · 10 Minutes

Testing Pyramid · @ParameterizedTest · @WebMvcTest scope · ArgumentCaptor · @Spy vs @Mock · TestContainers lifecycle

Homework & Checkpoint Commit

DEFINITION OF DONE — SESSION 10

- ProductServiceTest: @ParameterizedTest + ArgumentCaptor — green
- ProductControllerTest: 4 @WebMvcTest cases — GET 200/404, POST 201/400
- ProductRepositoryIntegrationTest: 2 TestContainers tests — green
- mvn test — zero failures, zero skipped
- docker compose up -d still works after test run (no port leaks)
- Commit: session-10: add-unit-and-integration-tests

KNOWLEDGE CHECK — BEFORE SESSION 11

Topic: Consumer-Driven Contract Testing

- What is the difference between a consumer and a provider in Pact?
- Why does a provider test pass even when the consumer would fail?
- What is a pact file and who owns it?

Bonus: read how WireMock compares to Pact — what problem does each solve?

5 Real Errors, 5 Real Fixes

Issue	Solution
Cannot find symbol: MockMvc / @WebMvcTest	Missing import spring-boot-starter-test in pom.xml, or wrong test scope. Check: <code><scope>test</scope></code> .
H2 tests pass, PostgreSQL fails in production	You hit a PG-specific type. Switch that test class to <code>@AutoConfigureTestDatabase(replace=NONE) + TestContainers</code> .
TestContainers: Could not find a valid Docker environment	Docker Desktop not running, or DOCKER_HOST not set in CI. Verify: <code>docker ps</code> from the build agent terminal.
ArgumentCaptor captures null / wrong value	<code>capture()</code> must appear INSIDE the <code>verify()</code> call: <code>verify(repo).save(captor.capture())</code> . After verify, call <code>captor.getValue()</code> .
@DataJpaTest loads services / fails with No qualifying bean	<code>@DataJpaTest</code> only loads JPA layer. Don't <code>@Autowired</code> a <code>@Service</code> here — use <code>@Import(MyService.class)</code> or move to a wider slice.



SESSION 10 COMPLETE

Tests at the Right Layer

Unit (JUnit 5 + Mockito) · @WebMvcTest · TestContainers + real PostgreSQL

NEXT SESSION

Session 11 — Contract Testing (Pact) + Chaos Engineering

WHEN

Wednesday, 3:00–5:30 PM